7BUISO22W Cyber Security Applications

Web Exploitation and Mitigation of OWASP Vulnerabilities – XSS and CSRF

University Of Westminster

Dr. David Huang

Learning Objective

1. The objective of this lab is to equip students with practical, hands-on experience in actively exploiting common OWASP Top 10 vulnerabilities

   - Cross-Site Scripting (XSS) is now primarily covered under A05:2025 – Injection. Although its ranking has decreased, it remains a widespread threat, with approximately 50% of web applications considered vulnerable. Cross-Site Scripting (XSS) remains the most frequently reported vulnerability pattern in CVEs, allowing attackers to inject malicious scripts into web pages.

   - Cross-Site Request Forgery (CSRF) is now commonly grouped under A01:2025 – Broken Access Control. Cross-Site Request Forgery (CSRF) is a common, high-risk attack that forces authenticated users to perform unauthorized actions without their consent. It continues to be a persistent threat and is often exploited in combination with other vulnerabilities to perform unauthorized actions.

➢ If you are working on your own machine:

- You must use Burp Suite Community Edition.
- You must have DVWA and WebGoat running on your Kali Linux or DVWA on OWASP virtual machine. Ensure that both applications are up and running before starting the lab.

➢ If you are working on a CLG.43/45 machine:

- Use the pre-installed Burp Suite Professional Edition and Docker software.
- Ensure that DVWA and WebGoat are configured in Host-Only mode and running on your Kali Linux machine, with proper communication established with the OWASP virtual machine.

For your assignment, you need to set up the assignment scenario on Kali Linux.

Topics Covered

- Cross-Site Scripting (XSS) and Reflected XSS
  - How to identify reflected XSS vulnerabilities and understand the potential impact of reflected XSS attacks.
  - Techniques for preventing XSS vulnerabilities, including the use of various common technologies and best practices for protecting against XSS attacks.
- Cross-Site Request Forgery (CSRF): Definition, common CSRF vulnerabilities, and methods for preventing CSRF attacks.
- Mitigation strategies and best practices for defending against both XSS and CSRF attacks.

Note: Some of these vulnerabilities and tasks may apply to either OWASP DVWA or WebGoat.

Safety and Ethics Note

- This lab uses intentionally vulnerable systems solely for educational purposes.
- All activities must be conducted within the local lab virtual machine (VM) environment. Testing against external or production systems is strictly prohibited.

Lab Tasks

1. Lab Eenvironment Setup

In this lab, we will use two vulnerable web applications:

- **Damn Vulnerable Web Application (DVWA)**

  DVWA is a deliberately vulnerable web application designed for security testing practice. On the left-hand side of the interface, you will see a menu containing links to all the vulnerabilities available for practice within the application.

  You must use only the DVWA running on the OWASP Virtual Machine. Access DVWA via one of the following methods:

  http://192.168.56.101/dvwa/

  OR

  Navigate to http://192.168.56.101 and select Damn Vulnerable Web Application

  Login credentials (OWASP VM DVWA):

  - username**:** **admin**
  - password: **admin**

- **OWASP WebGoat**

  OWASP WebGoat is a deliberately vulnerable web application that covers several OWASP vulnerabilities through structured lessons and interactive challenges. It is designed to help students understand and assess their knowledge of common web security issues.

  The application is available as a Docker container on Kali Linux. The WebGoat Docker container is already running on port 8082.

  You can access WebGoat using one of the following methods:

  http://127.0.0.1:8082/WebGoat

  OR

  http://webgoat.local

  Login credentials (Docker WebGoat on Kali):

  - username**:** **webgoat**
  - password: **webgoat**

2. Pre-Lab Learning Activities: XSS and CSRF (Burp Suite Web Academy)

Before starting the lab session, students are required to complete the following preparation tasks from Burp Suite Academy:

1. Study XXS and CSRF

Visit the following learning pages:

- **XSS (Cross-Site Scripting):** https://portswigger.net/web-security/cross-site-scripting
- **CSRF (Cross-Site Request Forgery):** https://portswigger.net/web-security/csrf

Learn and understand:

- What XXS and CSRF attacks are
- How to identify and exploit different types of XXS and CSRF vulnerabilities
- How to prevent XXS and CSRF attacks

2. Pre-Class Activities

Students must ensure they have an active account on PortSwigger.net. Registration is free and grants full access to the Web Security Academy.

Before attending the lab, review and understand the vulnerabilities covered in the **XSS** and **CSRF** sections of Burp Suite Academy. Familiarize yourself with these attacks, their associated vulnerabilities, and different types by completing the following related labs:

Such as:

- Reflected cross-site scripting (or XSS) occurs when an application receives data in an HTTP request and includes it in the immediate response in an unsafe manner.
- Stored cross-site scripting (also known as second-order or persistent XSS) Occurs when an application receives data from an untrusted source and includes it in later HTTP responses in an unsafe manner.
- Basic SSRF against the local server check feature which fetches data from an internal system.
- CSRF vulnerability with no defenses In this scenario, an attacker can craft HTML that performs a CSRF attack, such as changing a user's email address, and upload it to an exploit server to execute the attack.
- Server-side request forgery (SSRF) a web security vulnerability that allows an attacker to cause the server-side application to make requests to an unintended location.

These exercises will help you gain hands-on familiarity with techniques and prepare you for the practical lab activities using DVWA and WebGoat.

3. In class Lab Tasks:XXS and CSRF : Exploitation and Mitigation Using DVWA and WebGoat

Part 1 Cross-Site Scripting (XSS) in DVWA or WebGoat

To identify and exploit a **Reflected Cross-Site Scripting (XSS)** vulnerability using **Damn Vulnerable Web Application** running on the **OWASP Broken Web Applications** virtual machine.

A. Observing Normal Application Behaviour

Start the OWASP VM, open a browser inside it, log in to DVWA using the OWASP VM credentials, and from the left-hand menu select XSS (Reflected).

Submit Normal Input : Enter a normal name (e.g., `John`) into the input field, then Click **Submit**.

Observation

- The application searches for the entered name.
- If no match is found, it returns a message such as *"No results found."*
- The input appears in the response page.

This establishes the application's **normal behaviour**.

Testing for Input Reflection

Enter Special Characters

Now test how the application handles unexpected input.

Enter:

```
<'this is my XSS test'>
```

Click **Submit**.

Observation

- The exact input is displayed in the response.
- The application reflects whatever is typed into the input field.
- The input becomes part of the HTML response.

This suggests that the application may be vulnerable to XSS.

Viewing the Page Source

Right-click anywhere on the page. Select **View Page Source**.

Analysis

Observe how your input appears inside the HTML structure.
If the input is inserted directly into the page without encoding, it may allow script execution.

B. Attempting Script Injection

Inject JavaScript Code

Enter the following payload:

```
<script>alert('Vulnerable to XSS')</script>
```

Click **Submit**.

Expected Result

If the application is vulnerable:

- A JavaScript alert box appears.
- The message *"Vulnerable to XSS"* is displayed.
- The script executes in the browser.

This confirms a **Reflected XSS vulnerability**.

C. Analysing the Input Using Burp Suite

Let us examine what happens to the input when it is submitted.

To do this, we will use **Burp Suite**, a powerful web application security testing tool. Before using **Burp Suite**, ensure the browser is configured to use it as a proxy and that traffic interception is enabled, then return to the XSS challenge and resubmit the payload.

<script>alert('Vulnerable to XSS')</script>

You may notice a different result. The application appears to be filtering the input, but it is not immediately clear what is being filtered. This does not necessarily mean the application is secure; it simply indicates that some input sanitisation is taking place.

Viewing the Intercepted Request

1. In Burp Suite, go to the **Proxy** tab.
2. Select **HTTP history**.
3. Locate the POST request related to the XSS challenge.
4. Select the POST message to inspect its contents.

You will see the submitted input within the request parameters.

Identifying the Filter

Upon inspection, you will observe that the word script has been modified. Specifically, the letter **"i"** has been removed, causing <script> to become <scrpt>. As a result, the browser does not recognise it as a valid HTML tag, and the script does not execute.

This shows that the developers attempted to prevent XSS attacks by filtering the word script — however, this approach is insufficient.

Attempting to Bypass the Filter

Since the application filters the word script, we can attempt to inject another HTML element that does not contain this word:

```
<IMG SRC="#" ONERROR="alert('Vulnerable to XSS')" />
```
or

```
""><svg onload=alert(1)>"
```

This payload works as follows:

- The image source # is invalid, so the image fails to load.
- The ONERROR event is triggered.
- The JavaScript code inside the ON ERROR attribute executes.
- An alert box displaying "Vulnerable to XSS" appears.

This confirms that the XSS injection was successful despite the filtering mechanism. You can also review both the request and response in the corresponding POST message.

Use Burp Suite to intercept the request and examine how the submitted input is processed and modified by the application before it is rendered in the browser (Figure. 1)

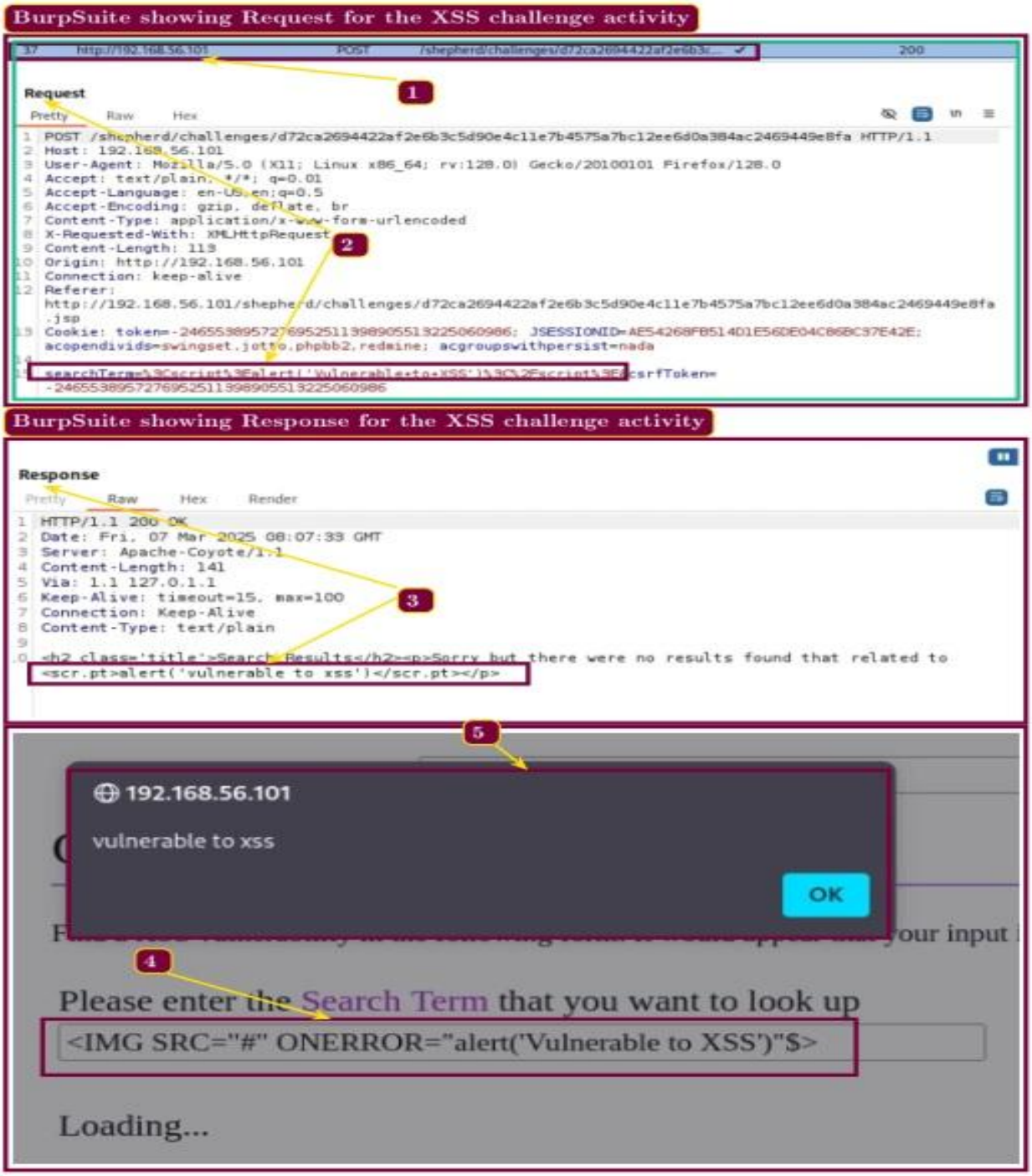

Figure 1. Analysing How XSS Input Is Processed Using Burp Suite

Part 2 Cross-Site Request Forgery (CSRF) in DVWA

Cross-Site Request Forgery (CSRF) is a type of attack that tricks a web browser into executing an unwanted action on a different site for which the user is authenticated. This repository serves as an educational resource for understanding the mechanics of CSRF attacks and the measures to prevent them.

To understand how Cross-Site Request Forgery (CSRF) works and to exploit a CSRF vulnerability in Damn Vulnerable Web Application.

A. Understanding How CSRF Works

Cross-Site Request Forgery (CSRF) is an attack that forces an authenticated user to perform unintended actions on a web application.

1. **User Authentication:** The user logs in to a website and receives an authentication cookie, which the browser automatically includes in future requests.
2. **Attack Setup:** An attacker creates a malicious webpage or link containing a forged request (e.g., to change a password).
3. **User Interaction:** While still logged in, the victim visits the malicious page or clicks the link.
4. **Request Execution:** The browser sends the forged request along with the user's authentication cookie, and the server processes it as if it were a legitimate action.

B. Part Exploiting CSRF in DVWA (Low Security Level)

To identify and exploit a CSRF vulnerability using Damn Vulnerable Web Application running on the OWASP Broken Web Applications virtual machine.

Cross-Site Request Forgery (CSRF) is an attack that forces an authenticated user to perform unintended actions on a web application.

It works because:

- The user logs in and receives a session cookie.
- The browser automatically includes this cookie in future requests.
- The server cannot distinguish between legitimate and forged requests.

Step 1 – Configure DVWA
Log in to DVWA, set the security level to **Low**, and select **CSRF** from the left-hand menu.

Step 2 – Observe Normal Functionality

1. Enter a new password (e.g., 12345).
2. Click **Change**.

Observe that:

- The password is changed successfully.
- The new password appears in the URL:
  .../vulnerabilities/csrf/?password_new=12345&password_conf=12345&Change=Change

This indicates:

- The request uses HTTP GET.
- Sensitive data is exposed in the URL.
- No CSRF protection is implemented.

Step 3 – Craft a CSRF Attack Page

Create a file named attack.html with the following content:

```
<html>
  <body>
    <a
href="http://localhost/dvwa/vulnerabilities/csrf/?password_new=hacked123&
password_conf=hacked123&Change=Change">
      Click here
    </a>
  </body>
</html>
```

(Modify the URL if required.)

Step 4 – Execute the Attack

While logged in as **admin**, open attack.html and click the link; the browser sends the request with the session cookie, and the password is changed without further verification.

Step 5 – Verify the Exploit

- Log out of DVWA.
- Log in using:
  - Username: admin
  - Password: hacked123

If login succeeds, the attack was successful.

C. Analysis

At the Low security level:

- No CSRF token is implemented.
- No origin or referrer validation is performed.
- The application trusts any request with a valid session cookie.

This demonstrates that authentication alone does not prevent CSRF attacks.

This lab demonstrates how CSRF exploits the trust between a browser and a web application. Proper defensive mechanisms such as CSRF tokens, POST requests for sensitive actions, and origin validation are necessary to prevent such vulnerabilities.

- **How XXS and CSRF violate the CIA Triad**

  The lab experiments demonstrate how attackers can compromise system security through XSS and CSRF attacks.

  ### XSS (Cross-Site Scripting)

  XSS can affect all three components of the CIA triad, with the strongest impact on confidentiality and integrity.

  - **Confidentiality:** Steals cookies, tokens, and sensitive page data
  - **Integrity:** Modifies webpage content or form data
  - **Availability:** Can crash or freeze web pages

  ### CSRF (Cross-Site Request Forgery)

  CSRF mainly violates integrity.

  - **Confidentiality:** Indirect impact (e.g., changing email or password)
  - **Integrity:** Performs unauthorized actions (primary impact)
  - **Availability:** Can delete data or lock users out

  XSS = Running malicious code in the victim's browser

  CSRF = Forcing the victim to perform unwanted actions

  These activities highlight the severity of these vulnerabilities and their potential impact on the **Confidentiality, Integrity, and Availability (CIA)** of a system.

4. Post-Lab Questions

1. What are XSS and CSRF attacks?
2. Why is using GET insecure for sensitive actions?
3. How can attackers bypass simple input validation?
4. Why did the `<script>` payload work at Low security in DVWA?
5. Why is keyword filtering (e.g., `script`) not enough?
6. What role does the browser play in executing XSS?
7. How can XSS be prevented in web applications?
8. How do CSRF tokens prevent attacks?
9. What coding practices help prevent XSS and CSRF?
10. What are the real-world impacts of XSS and CSRF?
11. What other security measures could be implemented?

5. Mitigation Techniques for XSS and CSRF

These mitigation techniques help protect the system's Confidentiality, Integrity, and Availability (CIA) by preventing unauthorized data access and actions.

XSS (Cross-Site Scripting) Mitigation

- **Input validation** – Validate and sanitize all user input
- **Output encoding** – Encode data before rendering it in HTML, JavaScript, or URLs
- **Content Security Policy (CSP)** – Restrict execution of unauthorized scripts
- **HttpOnly cookies** – Prevent JavaScript from accessing session cookies
- **Secure frameworks** – Use modern frameworks that automatically escape output

CSRF (Cross-Site Request Forgery) Mitigation

- **CSRF tokens** – Include unique, unpredictable tokens in forms
- **SameSite cookies** – Prevent cookies from being sent with cross-site requests
- **Re-authentication** – Require password confirmation for sensitive actions
- **Check HTTP Referer/Origin headers** – Verify the request source
- **Use POST instead of GET** for state-changing actions

Lab Journal and Report Collection for Lab-Based Portfolio Assessment

You are required to prepare a lab report, including screenshots, that clearly describes what you have done and what you have observed. This report will help you build a lab journal for the lab-based portfolio assessment.